Service Oriented Computing and Applications (2020) 14:283–296
https://doi.org/10.1007/s11761-020-00301-1

ORIGINAL RESEARCH PAPER



Semantics-based API discovery, matching and composition with linked metadata

Diego Serrano[1] · **Eleni Stroulia**[1]

Received: 21 March 2020 / Revised: 24 August 2020 / Accepted: 28 August 2020 / Published online: 22 September 2020
© Springer-Verlag London Ltd., part of Springer Nature 2020

Abstract
Web APIs have been adopted as the de facto standard for exchanging data on the Web. However, engineering applications that orchestrate the invocation of multiple APIs and the data flow among them are still mostly manual and labor intensive. In fact, as the number of the potentially relevant APIs increases, compositions become opaque, difficult to maintain, and practically impossible to reuse. The recent advances around linked data formalisms have the potential to provide "usable" semantics, to enable automatic API composition methods. In this paper, we formalize a simplified description model, based on SPARQL graph patterns, for capturing the semantics of Web APIs. Based on this model, we propose a methodology for a fully automated process that produces semantically valid composition chains, using iterative subgraph isomorphism. We have validated the usefulness and accuracy of our approach, using a collection of publicly available Web APIs relevant to a real-world use cases.

Keywords Semantic Web services · Automatic service composition · REST · SWAGGER · SPARQL · RDF · Linked data

1 Introduction

The ever increasing number of Web services has provided unprecedented interoperability, by exposing interfaces of invocable services that exchange data in formats such as XML and JSON. The number of Web APIs has increased from only 250 APIs in 2006 to more than 18,000 in 2017,[1] which has enabled companies to increase their revenues by discovering and joining forces with complementors.

In today's highly distributed, multi-platform world, there is a huge potential for creating added value through automated service composition. However, without a semantic specification of the APIs and the data they require and provide, machines cannot automatically interpret and appropriately route them. Thus, developing applications that rely on Web APIs still requires a considerable effort by developers, who still have to identify relevant APIs, write code for invoking them, and transform the data in conformance to their application structure.

In parallel with the advancement in Web service technologies, the development of semantic representation languages has driven the creation of promising knowledge description formalisms. Some remarkable examples are the linking open data (LOD) project, which has more than 1000 interlinked datasets in a variety of areas [46], and *schema.org*, that provides a vocabulary used by search engines to integrate structured data markup contributed by Web sites [15]. Although these projects demonstrate the potential of linked data approaches for data integration, the absence of APIs to regulate their access makes them inapplicable for most commercial applications and precludes business adoption.

In order to support automated service composition and coordination, the semantic Web envisioned general-purpose agents to automatically discover, compose, and act on Web services. However, after many years of effort, there is no generally accepted formalism for describing the semantics of Web services, or practical software-engineering frameworks making use of such descriptions, because of the complexity of the proposed description formalisms and their limited coordination functionalities.

Our approach, Linked REST APIs, overcomes these obstacles with a middleware for REST-API integration that

[1] As reported by ProgrammableWeb.com.

✉ Eleni Stroulia
stroulia@ualberta.ca

Diego Serrano
serranos@ualberta.ca

[1] Department of Computing Science, University of Alberta, 2-32 Athabasca Hall, Edmonton, Canada

aligns these two widely adopted technologies, Web APIs, as the preferred standard for exchanging data on the Web, and linked data vocabularies, as the mechanism enabling the automated semantic integration of services. In our previous work [47], we described the intuition behind the overall architecture of our middleware and composition methodology. In this paper, we present a detailed formalization of the description semantics and elaborate on the composition and matchmaking definitions that enable our middleware to automatically regulate the composition and enactment process. The paper presents three main contributions:

1. A *detailed formal model for capturing the semantics of REST APIs*, and a comprehensive methodology for fully automated service composition.
2. A *semantically annotated dataset* extracted from real service offerings, representative of the variety in terms of API design patterns and resources found in real environments.
3. A *comprehensive empirical evaluation of LRA*, demonstrating our method is accurate, in that it returns (most of) the service composition chains that experts in the domain would develop, and efficient, in that it improves the execution time when compared to similar approaches

The rest of the paper is organized as follows. Section 2 reviews the evolution of the work around semantic Web services. Section 3 presents an example that illustrates how different APIs may converge in our integration middleware. Section 4 explains the LRA formalization. Section 5 describes the LRA methodology for identifying Web services that can potentially be used in a composition. Section 6 discusses the LRA method for automatic service composition. Section 7 discusses the architecture of the LRA middleware. Section 8 presents the results of our evaluation. Section 9 concludes with a statement of our contributions and plans for future work.

2 Review of semantic service composition

In order to support automated discovery, composition, and enactment of Web services, a method must include (a) a *formal description* of the functional semantics of the service, and (b) *algorithms* that leverage these descriptions for the aforementioned tasks.

OWL-S [32] and WSMO [11] proposed formal descriptions of the IOPE (inputs, outputs, preconditions, and effects) signature of services, and, sometimes, their non-functional properties. OWLS-MX [23], WSColab [14], IRS-III [5], and the algorithms of Paolucci et al. [37], Jaeger et al. [21], and Huang et al. [20] were built on these languages; however, they were not broadly used because of their complex representation assumptions.

DIANE [27] defines constraints on the input values; service offers are described as the set of effects they can provide, and service requests are described as the set of effects that are acceptable for the requester. The DSD matchmaker, however, is limited to parallel composition patterns only.

In response to the complexity and limited use patterns of previous approaches, METEOR-S [38] specified workflow templates where concrete services are bound to each workflow node at runtime; SWSComposer [19] relies on SAWSDL [25] specifications to construct approximate compositions; and ComposIT [43] proposes a graph-based service composition based on input–output parameter matching of services' interfaces. These composition algorithms, however, based on lightweight descriptions are susceptible to incorrect solutions.

WSMO-Lite [44] introduced an ontology for service semantics that fits directly into SAWSDL annotations. However, its requirement for specifications to conform to general-purpose abstract functional classifications fundamentally limits its applicability.

More recently, LIDS [48], LIDSEARCH [35], RESTdesc [53], Karma [49], and SPARQL Micro-Services [33,34] used RDF and SPARQL, to describe semantic mappings between the mediating schema and the local sources. LIDSEARCH takes requests as SPARQL queries and extracts input and output information to discover relevant services, but is unable to support composition. The linked data services (LIDS) approach is more sophisticated and proposes a description of Web services using RDF and SPARQL graph patterns, describing inputs and outputs as SPARQL strings; this allows for inconsistencies between the SPARQL expressions and precludes automated discovery and composition. Karma and RESTdesc adopt logic-based specifications, SWRL and N3, respectively, that are verbose and complex and, again, hinder adoption in real-world software.

SPARQL Micro-Services use SPARQL queries that can act on service responses, translating them to RDF triples. However, SPARQL Micro-Services behave only as a data wrapper that loads a parsed response into an temporary RDF graph and executes the query on such graph, without defining a matchmaking mechanism. Unfortunately, the aforementioned approaches are simply assumed to be complete and correct, only illustrated by examples, without experimental evaluation. Other recent approaches, such as Triple Pattern Fragments [41] and SmartWS [30], were not considered, as they assume services that consume and produce RDF, which are rarely found in the industry, and their proposed composition is performed at the data level, not at the service level.

Facebook's GraphQL [10] is a query language for APIs, created by defining types and fields, using a domain-specific language, and then providing functions to retrieve the data for

each field on each type. The execution procedure in GraphQL is defined by the recursive execution of each field, which is backed by a function. However, GraphQL is fairly simplistic in that it only represents data using a syntactic schema (without semantics), and the composition only supports resources that adhere to a very restrictive REST style specification.

Despite their prevalence in the literature, none of the aforementioned semantic Web services proposals were supported by a working middleware with enough traction to drive the development of a community, which led to their abandonment. Some authors acknowledged the difficulties of interpreting standards and defining semantic Web services and have created tools to support users in annotating Web services, such as MWSAF [38] and ASSAM [17]. However, these tools also suffer from a lack of broad adoption, due to the unpopularity of semantic Web services, thus experiencing a classic chicken-and-egg problem.

3 Scenario and motivation

An example that demonstrates the need for automated service integration systems can be seen in the application domain of research program evaluation. Digital libraries and funding agencies offer Web APIs to enable access to metadata of millions of publications and grants; however, none of them contain all the publications of an individual author nor do they contain all the information relevant to program budgets assigned to researchers.

In order to illustrate our approach, we consider a typical scenario that involves collecting the publications of a particular researcher. We will assume there are two relevant service providers, but fictional, *MCA* and *EEEI*, and the developer only knows the name of the author. The information managed by the two service providers is similar, but has differences in coverage, response structure, and interface. In the rest of this paper, we will use a set of operations from each service provider, exposing data through REST services. *MCA* has one operation: `getPapers`, which receives as an input a person's email and returns a list of academic papers created by the person. And *EEEI* has two operations: `searchAuthors`, which receives as an input a person name and returns a set of authors having that specific name; and `getDocuments`, which returns a list of academic papers created by an author, whose identifier is received as an input. This scenario highlights our motivation, since it can show how an input that is seemingly incompatible with the paper-related service operations can be transformed into a service composition task to find semantically linked resources that can be used, in conjunction, to achieve the goal. In the following sections, we describe how our approach is used to describe and compose semantically linked resources on the Web.

4 The LRA formal model

Service composition methods aim at combining several service operations to fulfill a request. For example, to retrieve the titles of a person's publications, when there is no a single service operation that can achieve such a request on its own, a composition method may combine operations that return publication DOIs given a researcher's name and publications titles given a DOI.

The foundation for automatic service composition is the semantic description of the underlying resources. This section builds upon our previous work [47], by formalizing the data model for representing information about the resources, and the mechanism to provide access to such data.

LRAs[2] are described by the schema of the data they serve, expressed as SPARQL graph patterns. LRAs introduce a simple vocabulary for the semantic representation of functional and non-functional attributes of Web APIs; this vocabulary incorporates the structural organization of widely adopted syntactic API description formats, such as OpenAPI and RAML, thus simplifying their integration in the LRA middleware. The vocabulary is guided by two of the design principles defined for the minimal service model in WSMO-Lite [44]: *proximity to underlying standards*, in this case OpenAPI and RAML, and *lightweight minimality*, adding only very few new constructs on the top of well-known underlying technologies.

The vocabulary defines a *Service* element that acts as the container of the resources and operations of a service provider. Each *Service* offers a number of *Operations*, which, in turn, are associated with graph patterns that represent their underlying data schema, and the links between input and output elements. Figure 1 shows a diagrammatic representation of the global graph pattern formed by the combination of all the operations in our motivating example.

Definition 1 *(Linked REST APIs)* LRAs are specified as a set of operations, where each operation S is defined by a 6-tuple $S = (E, P, I, O, A, T)$ of endpoint information, a graph pattern, inputs, outputs, authentication mechanisms, and quality attributes.

Consider for example the `getPapers` operation. The endpoint $E = \{urlp, method, type\}$ denotes the set of grounding parameters of the service, namely the URL pattern (similar to URI templates in hRESTS [44]), the HTTP request method, and the type of operation. This type of operation is based on Hydra [29] and is used to indicate if an operation results in resources being read, created, deleted, or replaced. In this example, the request method is *GET*, the type is *ReadAction*, and the URL pattern is defined by

[2] LRA specifications of Web APIs.

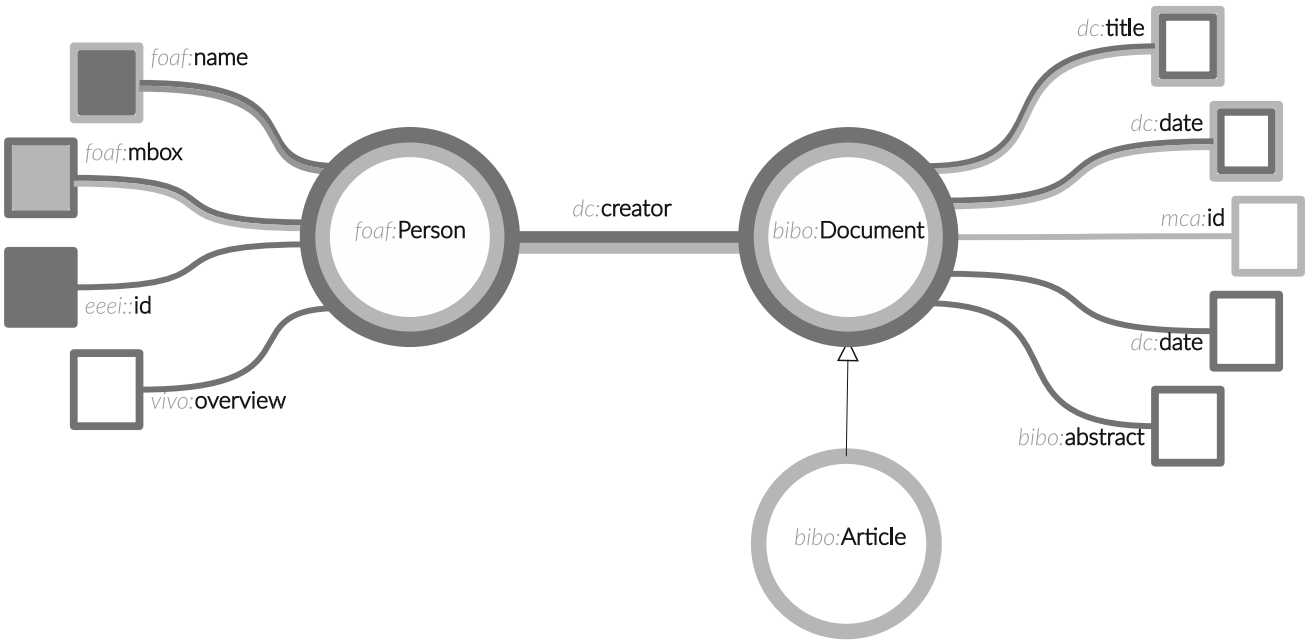


Fig. 1 Global graph representation: Green and blue denote entities and attributes supplied by MCA and EEEI; fill colors represent the data provider that uses the attribute as input

http://api.mca.com/persons/papers?email={email}, where the email is meant to be replaced with an input value.

P defines the graph pattern, composed of triple patterns, representing the data managed by the operation. The operation `getPapers` relates persons to their articles, exposing information about the name of the person, and the titles, identifier numbers, and publication dates of the articles. In particular, the SPARQL graph pattern of `getPapers` is expressed as:

```
?person a foaf:Person ;
      foaf:mbox ?email;
      foaf:name ?name .
?article a bibo:Article ;
      dc:title ?title;
      mca:id ?id;
      dc:date ?date ;
      dc:creator ?person .
```

I is a partial function $I : \mathsf{nodes}(P) \rightarrow \{itype, iname\}$ mapping nodes (elements found in subjects and objects) of the graph pattern to their associated type of input and grounding name. The type of input is defined by establishing the cardinality (required or optional), and the type of match (full or partial, specially for search functionalities that allow to specify a portion of the term). In the example operation `getPapers`, there is only one input, the email, which is required, full match, and is mapped to the url substitution pattern `{email}`.

O is a partial function $O : \mathsf{nodes}(P) \rightarrow path$ mapping nodes in the graph pattern to their associated location path, represented as an XPath-like expression. In the operation `getPapers`, the title of the documents `?title` is mapped to the expression `/papers/title`, which indicates that the titles are located in the title value, within the papers object in the response document.

A denotes the set of parameters used by the authentication mechanism of the operation, which will vary depending on the authentication protocol.

T is a mapping function $T : metric \rightarrow \mathbb{R}$ that defines associations between quality metrics of the operation, such as availability or latency, with its associated value. This information can be used to discard composition chains with operations below some thresholds.

5 Service matchmaking

In order to leverage the LRA semantic descriptions, SPARQL is used to express a service composition request. Therefore, the SPARQL query graph pattern is used to identify service operations that can provide all or part of the answer, by matching subgraphs of the query graph against subgraphs of the graph patterns defined in the LRA operations.

In principle, SPARQL queries provide an equivalent expressive power to relational algebra [1]. Perez et al. [39] demonstrated that the complexity of evaluation of query patterns constructed by using concatenation, FILTER, and UNION operators is NP-complete, and when OPTIONAL operators are included, the complexity is PSPACE-complete. At the same time, restricting operators and imposing restrictions on optional blocks, in *well-designed graph patterns*, yield more efficient query evaluations. Our approach restricts

the queries to well-designed graph patterns, which are very common in practice, and simplifies the execution of the query plans. More specifically, LRA SPARQL queries are union-free graph patterns, restricted to a single optional block.

Definition 2 *(Query graph pattern)* A query graph pattern expression for LRA, representing a declarative service composition request, is defined as a conjunction of a basic graph pattern, denoted by Q_R, and an optional basic graph pattern, denoted by Q_O. Then, using SPARQL's grouping syntax, $Q = Q_R$ OPT$\{Q_O\}$.

Therefore, a query from our motivating example, which retrieves the titles and (optionally) publication dates of the documents authored by "John Smith" (a fictional name), can be represented as the following query:

```
Q:   SELECT ?title ?date
     WHERE { ?author a foaf:Person ;
                foaf:name "John Smith".
             ?doc a bibo:Document ;
               dc:creator ?author ;
               dc:title ?title
     OPTIONAL { ?doc dc:date ?date } }
```

Given a query, the first step toward a service composition is to identify service operations that may provide relevant information, by finding the operations whose graph patterns can be mapped, at least partially, on the query pattern. Such a mapping is defined based on the equivalence of triple pairs. Informally, the concept of triple equivalence implies that, even though a triple may not have an exact counterpart, an equivalent triple can be established based on inference rules. For instance, a query triple `?x a bibo:Document` can be mapped to a triple `?x a bibo:Article`, since `bibo:Article` is subsumed by `bibo:Document`.

The basic inference rules in LRA are based on a subset of properties of RDFS, namely *subPropertyOf* (sp), *subClassOf* (sc), and *type* (type), and two properties of OWL, *equivalentClass* (ec) and *equivalentProperty* (ep). *subClassOf* and *subPropertyOf* are binary properties reflexive and transitive that, when combined with *type*, specify that the type of an individual or property can be lifted to that of a superclass or superproperty, respectively. In turn, *equivalentClass* (and *equivalentProperty*) states that two classes (or properties) have the same class (or property) extension. Formally, entailment properties are expressed as first order formulas:

$\forall c_1, c_2, c_3(\text{triple}(c_1, \text{sc}, c_2) \wedge \text{triple}(c_2, \text{sc}, c_3) \supset \text{triple}(c_1, \text{sc}, c_3))$
$\forall s, c_1, c_2(\text{triple}(s, \text{type}, c_1) \wedge \text{triple}(c_1, sc, c_2) \supset \text{triple}(s, \text{type}, c_2))$
$\forall c(\text{triple}(c, \text{type}, \text{Class}) \supset \text{triple}(c, \text{sc}, c))$
$\forall p_1, p_2, p_3(\text{triple}(p_1, \text{sp}, p_2) \wedge \text{triple}(p_2, \text{sp}, p_3) \supset \text{triple}(p_1, \text{sp}, p_3))$
$\forall s, p_1, p_2, o(\text{triple}(s, p_1, o) \wedge \text{triple}(p_1, sp, p_2) \supset \text{triple}(s, p_2, o))$
$\forall s, c_1, c_2(\text{triple}(s, \text{type}, c_1) \wedge \text{triple}(c_1, \text{ec}, c_2) \supset \text{triple}(s, \text{type}, c_2))$
$\forall s, p_1, p_2, o(\text{triple}(s, p_1, o) \wedge \text{triple}(p_1, \text{ep}, p_2) \supset \text{triple}(s, p_2, o))$
$\forall p(\text{triple}(p, \text{type}, \text{Property}) \supset \text{triple}(p, \text{sp}, p))$

There are three reasons why our work is restricted to these properties only. First, according to Tomaszuk [50] the selected properties account for around 40% of the most used terms at LOD. Second, the equivalence implied by the properties is relatively computationally simple. Finally, reasoning about instances (e.g., *owl:sameAs*) or enforcing constraints based on instances (e.g., *minCardinality*) is difficult because Web APIs typically do not allow full access to their data. For simplicity, we will designate the subsumption and equivalence properties with the symbol $\models$, which denotes that a triple can entail another triple and lays the foundation to formally define triple equivalences.

Definition 3 *(Triple equivalence)* A triple t' from a query graph pattern is considered equivalent to triple t from a service operation graph pattern, denoted by $t \equiv t'$, if $t' = t$ or $t \models t'$, where the particular variable names are ignored, and URIs and Literals of t' can be mapped to variables in t.

Hence, the mapping of a service operation to a query is defined as the set of equivalent triple pairs of their corresponding graph patterns, as follows.

Definition 4 *(Query graph mapping)* Given a query graph pattern Q, a mapping μ is a partial function $\mu : T_Q \rightarrow T_{S_i}$, which indicates how triples of the query T_Q are mapped to triples in an service operation graph pattern T_{S_i}. Two mappings μ_1 and μ_2 are considered compatible when for all $t \in \text{domain}(\mu_1) \cap \text{domain}(\mu_2)$, it is the case that $\mu_1(t) \equiv \mu_2(t)$, where $\text{domain}(\mu_i)$ extracts the domain, or query triples, of mapping μ_i.

Regarding mapping compatibility, if we consider $\mu_1 = \{(t_1, t_{s_1}), (t_2, t_{s_3})\}$ and $\mu_2 = \{(t_1, t_{s_1})\}$, they represent two compatible mappings because their overlapping triple mappings are equivalent. Similarly, if μ_2 adds a new triple mapping (t_4, t_{s_5}), the two mappings remain compatible because the new query triple is not covered by μ_1. On the contrary, if μ_2 is changed to $\mu_2 = \{(t_1, t_{s_2})\}$ the mappings are not compatible, because μ_1 relates t_1 to t_{s_1}, but μ_1 relates t_1 to a different triple t_{s_2}.

In order to represent the semantics of query–operation match evaluation, we adapt the relational algebra definitions of the operations of join and left join of two sets of mappings m_1 and m_2. Intuitively, $m_1 \bowtie m_2$ is the result of extending the mappings in m_1 with the compatible mappings in m_2; and $m_1 \bowtie\!\!\!\!\!\!\bowtie m_2$ extends the elements in $m_1 \bowtie m_2$ with the mappings in m_1 that are incompatible with the mapping of m_2, in order to support the optional operator.

$$m_1 \bowtie m_2 = \{\mu_1 \cup \mu 2 \mid \mu_1 \in m_1, \mu_2 \in m_2 \text{ , and } \mu_1 \text{ and } \mu 2 \text{ are compatible}\}$$

$$m_1 \bowtie\!\!\!\!\!\!\bowtie m_2 = m_1 \bowtie m_2 \cup \{\mu_1 \in m_1 \mid \forall \mu_2 \in m_2 \text{ , and } \mu_1 \text{ and } \mu_2 \text{ are incompatible}\}$$

Considering the definitions of query graph mappings and join operations, our approach determines whether an API

operation may provide data related to the query graph pattern, as follows.

Definition 5 *(Query–API subgraph match)* The evaluation of the subgraph of a query graph pattern Q, over a subgraph of a service graph pattern S_i, denoted by $[\![Q]\!]_{S_i}$, is defined recursively as follows:

1. If Q is a triple pattern t, then $[\![Q]\!]_{S_i} = \{\mu \mid \text{domain}(\mu) = t\}$.
2. If Q is (Q_1 AND Q_2), then $[\![Q]\!]_{S_i} = [\![Q_1]\!]_{S_i} \bowtie [\![Q_2]\!]_{S_i}$.
3. If Q is (Q_1 OPT Q_2), then $[\![Q]\!]_{S_i} = [\![Q_1]\!]_{S_i} \bowtie [\![Q_2]\!]_{S_i}$.

For example, considering the query presented previously, which retrieves the titles and publication dates of the documents authored by "John Smith," with respect to the `getPapers` operation, the following match is identified:

```
?author a foaf:Person → ?person a foaf:Person
?author foaf:name "John Smith" → ? person
    foaf:name ?name
?doc a bibo:Document → ?article a bibo:
    Article
?doc dc:creator ?author → ?article dc:creator
     ?person
?doc dc:title ?title → ?article dc:title ?
    title
?doc dc:date ?date → ?article dc:date ?date
```

Besides identifying query triples matching service operation triples, only the subsets of mappings produced by $[\![Q]\!]_{S_i}$ that are part of a connected component are considered to be a valid match, since mappings of disconnected components can lead to inconsistent compositions. A connected subgraph match of $[\![Q]\!]_{S_i}$ is a subset of mappings, such that triples $t \in \text{domain}([\![Q]\!]_{S_i})$ form a connected component, like in the previous subgraph match example.

Finally, unlike RDF data graphs, data from Web APIs are only accessible when a list of required inputs and authentication credentials is in place. Thus, in addition to being a connected subgraph match, the match needs to comply with the input interface of the API operation. For instance, although the query and the `getPapers` operation form a connected subgraph match, the service operation requires the email as the input, which is not provided in the query, thus causing the match to be incompatible. Then, in order to consider a Web API as relevant and accessible, the API graph pattern has to form a compatible subgraph match with respect to a subgraph pattern of the query.

Definition 6 *(Compatible subgraph match)* Given a service operation graph pattern S_i, a connected subgraph match is considered compatible if the node mapping function M, that relates query pattern nodes with their mapped service operation pattern nodes based on μ, satisfies that all the required input resources, denoted by $I^*_{S_i}$, have a mapping in M to a concrete value. More formally, when $\forall n, n \in \text{domain}(I^*_{S_i}) \wedge M(n) \in (U \cup L)$, where U and L denote URIs and Literals, respectively.

In summary, LRA defines a methodology to find semantically related Web API operations, based on isomorphic subgraphs, and input interface compatibility. In the next section, we will define an iterative algorithm to compose service operations, in order to fulfill the query requests.

6 Service composition

In the context of the LRA middleware, the service composition problem is formulated as that of finding a sequence of API operations $S = \langle S_1, \cdots, S_n \rangle$, with the following properties: **(1)** each service operation graph pattern has a non-empty mapping with the query graph pattern, and each required query triple is mapped by at least one of the operation triples; **(2)** the variables in the SELECT query are included as the output in one of the service operations; and **(3)** the service operations, when executed in sequence, form compatible subgraph matches. This implies that after each step in the composition chain, a new query, referred to hereinafter as the expanded graph, is created by extending the query graph pattern with the output triples of preceding service operation graph patterns, which at this point can be used as inputs.

In our example, since only the name of the author is available at the beginning, only `searchAuthors` is compatible and matches part of the input query. After `searchAuthors` is invoked, the composition algorithm can use the id and email returned by the operation in the next step of the composition and invoke the operations `getDocuments` or `getPapers`, which would both cover the initial query, resulting in two alternative composition chains.

This section extends the initial approach defined in [47], by detailing each composition step and describing the heuristic rules that enable an efficient execution. The composition of LRA operations consists of an iterative process, involving three steps: (a) *pairwise matching*, ensuring the semantics of the composition, and (b) *blocking*, and (c) *pruning* to enhance performance. These steps are repeated until *convergence* in a limited-depth exploration. As an final step, a set of *heuristic rules* refine the compositions and discard redundant composition chains.

Pairwise matching: At a high level, the composition problem can be represented as a graph search problem, where the global graph $G = (V, E)$ consists of a (finite) set of service operations denoted by V, and a collection E, of ordered pairs $\{u, v\}$ of distinct elements from V, where $\{u, v\}$ exists if the inputs in the current query that lead to u make v a compatible subgraph match of the query. After each node is visited, the expanded query is updated by adding the outputs of the current node.

Consequently, the basic step of service composition is pairwise matching, reusing the discovery methodology from the previous section and then deciding whether or not a service is a compatible subgraph match and can contribute to the overall goal. Our approach checks that the service is a compatible subgraph match, using an adaptation of Ullman's algorithm [52] that considers subsumption and equivalent triples. Subgraph isomorphism is a well-known NP-complete problem, and Ullman's backtracking algorithm exhibits exponential time complexity in the worst case.

In our example, due to compatibility, only `searchAuthors` is considered as the first node in the composition chain; and thanks to subsumption and equivalence, the concepts in the query (particularly, `bibo:Document`) are matched to the concepts in `getPapers` (which uses the concept `bibo:Article`).

Blocking: Pairwise matching until query coverage ensures the desired goal of service composition, but may be quite inefficient and even infeasible for a large set of services. The main cause of inefficiency is that pairwise matching may require an exponential number of comparisons to decide which subgraphs are isomorphic matches. Our method, inspired by the work of Garcia et al. [13], attempts to reduce the search space by using a blocking function that retrieves only the services that can use at least one of the inputs in the query. For example, an attempt to match the initial query that retrieves documents from "John Smith" with the operation `getPapers` would be unnecessary, since at that point, the email of the author is unknown, and thus, the operation cannot be invoked.

Pruning: As the global graph is being explored, some branches can be discarded without examining them. The pruning technique for LRA examines negative matches and passes on this information to its ancestor nodes, in order to avoid future exploration of those nodes, which cannot possibly influence the final decision, due to the extension principle of expanded queries. Consider the exploration of the composition chain $S = \langle a, b, c \rangle$, where only a and b but not c were successfully matched. This finding indicates to node a that, when using the same query, an exploration of a composition chain of the form $S' = \langle a, c, ... \rangle$ will fail. The general principle is that, as matches only add triples to the query (and possibly, flag nodes as outputs), if c cannot be matched to the original query plus the added triples from a and b, then it will not be matched to the original query plus the added triples from a, which is only a subset of the previous case.

Convergence: The exploration of the global graph of service operations may end up in an unbounded graph problem, preventing the comprehensive creation of composition chains for a query, if full traversals are used. Hence, our approach uses a depth-limited search that prevents impractical exploration by limiting the depth of the search to some value l.

Algorithm 1: LRA Composition

Global: An array of blacklisted operations for each depth level *pruning* (initially empty).
Input: A query graph Q, A finite set $R = \{S_1, S_2, ..., S_n\}$ of Web API operation graphs, a maximum exploration depth l, and a current depth *depth*.
Output: A list of composition chains S.

```
1  Function compose(Q, R, l, depth)
2      S = ∅
3      if l > max then
4          return S
5      while Q is not covered do
6          in = getInputs(Q)
7          operations = getCandidates(R, in)
8          for each S_i in operations do
9              if S_i ∈ ⋃_{i=level−1}^{depth} pruning[i] then
10                 continue
11             match = getMatch(Q, S_i)
12             if match covers new elements of Q then
13                 if match covers the rest of Q then
14                     add S_i operation to S
15                 else
16                     Q+ = Q + output of S_i
17                     next = compose(Q+, R, l, depth + 1)
18                     if next is not empty then
19                         prepend S_i to chains in next
20                         add S_i operation to S
21                 pruning[depth] = ∅
22             else
23                 add S_i operation to pruning[depth − 2]
24     return S
```

The implementation of the algorithm follows a recursive approach to dynamically compose the services, based on graph matching, as outlined in Algorithm 1. The process starts by extracting the triples, using Apache Jena, and identifying the potential inputs (line 5). The first composition step, described in the *blocking* section, aims at restricting the number of pairwise comparisons, by only considering service graphs that can use at least one of the inputs in the query graph (line 7). Then, the algorithm checks if the candidate operation is in the blacklist for the parent or successive operation levels, and, if so, discards it, implementing the *pruning* step (line 9). The next step uses subgraph isomorphism to determine whether the query graph is present within one of the service graphs, as explained in the *pairwise matching* section (line 11). As long as the match covers new elements of the query graph, the process keeps exploring composition chains from the current service (lines 12–21). Otherwise, the service operation is added to the blacklist of the grandparent level for future pruning (line 23). If the addition of the current service covers all the required elements of the query graph, then the service is added to the composition chain, and the algorithm stops exploration from that service (lines 13–14). The algorithm is applied recursively, creating an expanded query that integrates the output of previous services in the composition chain, until all the composition chains that cover all the required triples in the initial query graph are found within a given depth (lines 16–20). The algorithm exhibits an expo-

nential runtime complexity $O(n^l)$, where n is the number of service operations in the repository and l is the exploration depth

Refinement heuristics: The previous steps can produce a large number of composition chains that may be redundant or suboptimal [47]. The four heuristic rules, shown below, attempt to reduce the number of composition chains, while maintaining the same data coverage.

Duplicates chains are eliminated. Composition chains may include parallel executions of APIs independent of each other. When a composition exhibits commutative properties, including both $S_1 = \langle a, b\rangle$ and $S_2 = \langle b, a\rangle$, this rule filters out chains that are permutations of a previously processed chain, in this case discarding S_2.

Shorter chains are preferred. This rule discards the compositions that are superset of other chains, considering that shorter composition chains are more desirable because they preserve data semantics at a lower cost. For example, when two composition chains $S_1 = \langle a, b\rangle$ and $S_2 = \langle a, b, c\rangle$ are identified, then S_2 is discarded.

Chains involving fewer providers are preferred. This rule discards a composition chain that involves multiple service providers, if there are other shorter chains that already extract data from such providers. For example, assuming that services a and b are provided by R_1, and c is provided by R_2, if three composition chains $S_1 = \langle a\rangle, S_2 = \langle b, c\rangle$, and $S_3 = \langle c\rangle$ are identified, then S_2 is discarded, because the data from R_1 and R_2 can be extracted more directly with S_1 and S_3.

Chains where all providers are involved in shorter chains are eliminated. If a service provider already participates in one chain, there is no need to consider the provider in other chains, unless it is used to extract data from other provider that is not already part of one of the chains. For instance, assuming services a, b, and c are provided by R_1, and service d is provided by R_2, if we find three composition chains $S_1 = \langle a, b\rangle$, $S_2 = \langle c\rangle$, and $S_3 = \langle a, d\rangle$, then we can keep chains S_2 and S_3, because S_2 is the shortest composition that allows to extract data from R_1, and S_3, although is also using a service from R_1, is used as the only way to extract data from R_2, .

The aforementioned rules, except for the duplicate removal, may result in the elimination of desirable composition chains, which is shown in some queries of our evaluation. However, we found a better trade-off between redundancy and completeness when the rules are applied. Additionally, the use of each of the heuristic rules can be configured at the discretion of the developer, who can modify the relevant rules in the middleware configuration.

7 Composition enactment

The LRA middleware encompasses discovery, composition, and enactment of Web services, based on the methodology described in the previous sections.

Initially, the middleware is configured with a set of ontologies and service descriptions. The ontologies are loaded by the system and used to find triple equivalences, using the inference rules described in Sect. 5. For the example presented in Sect. 3, we load the ontologies *FOAF* [3], *Dublin Core Metadata Terms* [9] (using the prefix *dc*), and *BIBO* [8]. Then, following the definition of LRAs, as shown in Sect. 4 for the operation `getPapers`, the services are materialized by leveraging the OpenAPI specification,[3] which provides machine-readable interface files for describing, producing, and consuming RESTful Web services.[4] The LRA description defines five extension fields for OpenAPI: *prefixes* containing a list of ontology prefixes, *type* representing the type of operation, as defined in the endpoint information for LRA, *graphPattern* describing a SPARQL query pattern that represents the data managed by the service operation, *graphVariable* with associations of input parameters of the OpenAPI description and variables declared in the *graphPattern* expression, and *responseMap* defining a list of name–location pairs that associates output variable names in the *graphPattern* and response location paths that allow to lift output documents automatically.

The process starts with a declarative composition request, expressed as a SPARQL query, such as the query shown in Sect. 5 to retrieve titles and publication dates from a specific author. The query is then parsed, extracting the graph pattern and the parameters used as inputs and outputs. The query is passed to the composition engine, where the query graph pattern is matched against a service repository containing LRA service descriptions, following the procedure outlined in Sects. 5 and 6. After a set of composition chains have been identified, the access control and quality component discards some composition chains, by comparing the service authentication of the chains against the credentials provided by the developer. For example, if the operation `getDocuments` requires an API key which is not provided by the developer, then the composition chain `searchAuthors-getDocuments` is discarded.

Subsequently, the enactment component invokes the individual service elements of each chain. Once it receives the responses from the invoked API, the middleware *lifts* the data contained in the structured documents from its representation at the syntactic level into its corresponding semantic

[3] OpenAPI is an open-source collaborative project of the Linux Foundation. Originally known as the Swagger Specification.

[4] Examples of LRA descriptions used for the evaluation in Sect. 8 can be found at https://github.com/dfserrano/lraeval.

representation in RDF, relying on location path expressions, inspired by XPath. Location paths were informally presented in the example introduced in Sect. 4, with the location path expression `/papers/title`.

Additionally, in order to identify and merge records across a number of different data sources that refer to the same real-world entity, the middleware implements a simple clustering algorithm that considers accidental or deliberate errors, based on the work of Hernández and Stolfo [16].

8 Evaluation

In designing the evaluation method for our approach, we considered the test collections, summarized in Table 1, but none of these datasets were appropriate because of unavailability, incompatibility in their descriptions, or lack of reference requests that allows to evaluate the accuracy of our approach.

The service specifications of the OWLS-TC [24] collection do not express the relationships between input and output concepts, which is necessary for our method.

SWS-TC suffers from the same shortcoming as OWLS-TC; it only includes only input–output service specifications with no (or limited) semantic relations between them.

METEOR-S was one of the approaches evaluated with the Assam test collection, a set of 164 OWL-S service descriptions, curated by Hess et al. [18] from the now-extinct service listings *salcentral* and *xmethods*. It only provides semantic annotation for inputs and outputs, and it does not provide sample requests and reference response services.

The Jena Geographic Dataset (JGD) consists of 203 service operations, annotated with natural language documentation of the input and output parameters, as well as a description of the service functionality, with no relevance judgments.

The WSC challenge datasets were synthetically generated and do not have semantic information. The semantic Web service challenge (SWS) [40] dataset suffers from the same limitation.

After the season of semantic Web service competitions, the development of benchmarks has halted, which has especially affected the authors of recent semantic composition approaches that rely on semantics that expands the limits of simple signature annotations. This has forced them to validate their approaches on example use cases, which are often reverse-engineered from the solution.

In addition, the source code of other semantic service integration systems and the approaches that participated in the aforementioned challenges are not available.

We argue that a more realistic evaluation methodology is necessary, if we wish our methods to be adopted and have impact in practice. This is why we (a) created a new dataset and (b) designed an evaluation study around a realistic use case of software engineering teams building applications that integrate multiple Web services.

8.1 The LRA Web API collection

Our evaluation is designed around a realistic use case, as described by Brueckmann et al. [4], where a developer, who has previously identified a set of relevant service providers, is searching for a Web service, or a combination of services, that provides functionality needed in the application under development. This use case can be regarded as a special case of an information-retrieval problem, which is usually evaluated in terms of precision and recall. Accordingly, the evaluation involves three elements: a set of documents (represented by a set of service operations), a set of information needs (denoted by queries against the service operations), and a set of relevance judgments (represented by a set of composition chains that should be retrieved by the system for each query) [31].

Dataset: The service operation collection was extracted from real service offerings, in order to represent the variety in terms of design patterns and resources found in real environments. We selected the domain of scholarly APIs and used the list of APIs in https://www.library.ualberta.ca/about-us/open-data/api. Then, guided by an expert in the area, the number of service providers was narrowed down to 13 (52 operations) based on the relevance of the resources offered and the quality of the documentation. The relevant service operations were selected, removing the input parameters used for control and authentication, such as pagination parameters and API keys, in order to help relevance judges by eliminating noise that may interfere with the comprehension of the functional specification. It is important to note that the LRA middleware deals with these issues and supports service invocation and complete service-composition chain enactment.

The APIs were organized into a Web site, using Swagger UI, keeping the documentation content intact and allowing easy navigation for judges and developers during the evaluation (the APIs, questions, and relevance judgments are available at the Web site https://bit.ly/389ZiZH and the repo https://github.com/dfserrano/lraeval). The service providers are: arXiv, Australian National Data Service (ANDS), AltMetric, Connecting Repositories System (CORE), Crossref, Elsevier, IEEEXplore, Mendeley, Nature Blogs, National Science Foundation (NSF), ORCID, PatentsView, and PubMed E-Utilities. The test collection of services is described by the natural language documentation, in addition to their formal semantic descriptions, in order to allow the reuse in other approaches and avoid any bias that may be introduced by the abstraction process of creating the formal descriptions.

Queries: Complementing the collection of services, a librarian, with substantial expertise in research evaluation metrics and hands-on experience in producing research impact

Table 1 Overview of semantic Web services test collections

Name	Formalism	# Services	# Requests	Semantic scope	Last update	Available	Synthetic majority
OWLS-TC	OWL-S	1,083	42	Input/output*	2010	Yes	No
SAWSDL-TC	SAWSDL	1,080	42	Input/output	2010	Yes	No
WSC	Custom WSDL	3500+	5	Input/output	2010	No	Yes
SWS challenge	Natural language + WSDL	30	18	Natural language	2009	No	Yes
JGD	Natural language + WSDL	203	3	Natural language	2009	Only the services	No
SWS-TC	OWL-S	240	–	Input/output	2006	Yes	No
Assam	OWL-S	164	–	Input/output	2005	Yes	No

*OWLS-TC version 4 includes preconditions and effects for a subset of services (180 offers, 18 queries)

assessments, defined a set of representative queries of different levels of difficulty and complexity.

Relevance judgments: To evaluate the responses to the queries, the ground truth, namely the "desired service compositions" are defined for each query. By analyzing previous expert judgment approaches [28,37,47,51], we designed a clear and simple relevance model, based on *semantic correctness*, to determine whether the composition would yield results that are sound, and *usefulness*, to decide if the quality of the composition and its perceived cost make the composition worth it. Relevance is ranked in one of four levels: *SignificantMatch* (semantically correct, low or moderate cost, and represents the preferred option to extract data from the given providers), *PossibleMatch* (semantically correct, low or moderate cost, but is not the preferred option to extract data from the given providers), *InsignificantMatch* (semantically correct, but the composition is costly or is not expected to contribute a significant amount of valuable data), and *NoMatch* (semantically incorrect and ineffective). The first two ranks are considered *Relevant* and the last two are *Irrelevant*.

Four doctoral students of Computer Science and Information Studies from the University of Toronto, with relevant experience, served as relevance judges. The judges received a document with the eight natural language queries, the description of the relevance scale, and access to the Swagger interface with information about the available APIs. We gave the judges 2 hours to find and classify all the compositions that may answer each of the queries. Finally, we asked the judges to assess the compositions produced by the LRA method that were not included in their initial responses.

Service and query formalization: The VIVO ontology [6] is a unified, formal, and explicit specification of information about researchers, organizations, and the scholarly activities, outputs, and relationships that link them together. VIVO also promotes interoperability by reusing established linked data ontologies wherever possible, such as FOAF [3] and BIBO [8]. We therefore used it to represent semantic concepts in the domain of our evaluation. Based on the Swagger descriptions in YAML, and using the VIVO ontology [6], an expert in LRA extended the Swagger files with semantic annotations.

Table 2 Effort in LRA formalizations

Service provider	Operations	Effort (min.)
arXiv	1	26
ANDS	1	18
AltMetric	4	35
CORE	4	31
Crossref	6	40
Elsevier	13	65
IEEEXplore	1	10
Mendeley	4	22
Nature blogs	4	24
NSF	1	10
ORCID	9	47
PatentsView	1	13
PubMed	3	16

Table 2 shows the number of operations per service provider and the time spent by the expert in formalizing the semantics of the API. In average, the expert took almost 7 minutes to annotate the semantics of each operation, but as only one person created the descriptions, and there is significant potential to learn something from the ontology and similar API design patterns, the Web APIs described at a later stage are benefited from knowledge acquired by describing previous APIs, a phenomenon known as *learning effect* [45]. For example, since the APIs for *arXiv* and *IEEE* are similar, the time to describe the semantics of the latter is decreased considerably. In the future, we plan to incorporate approaches, such as the proposed by Mouhoub et al. [36], that can (semi-)automatically annotate the semantics of services.

Next, the natural language queries previously defined were encoded in SPARQL. In order to mimic realistic environments, the developer that expressed the queries did not use any information beyond the provided queries in natural lan-

guage, along with the ontology documentation, thus avoiding manipulation of queries to ensure correct retrieval.

In average, the developer spent 2.13 minutes per query, but the formulation of such queries also suffers from the *learning effect*. Table 3 in Sect. 8.3 shows the number of triples and joins used to represent the queries, which provide indication of the relative complexity. The characteristics of the queries are in line with the number of triples, joins, operators, and patterns found in the characterization of real-world queries extracted by Gallego et al. [12].

8.2 Results

The service annotations are fed into the LRA middleware, and queries are executed as expressed by the developer. This process results in lists of composition chains for each query, along with runtime and retrieval performance measures.

1: Which patents have been assigned to a person? In: Person name Out: Country, date, number, type, assignees name, title This is a high-selectivity query, since only one of the data sources provides information about patents. Accordingly, the experts identified only one composition chain, which was successfully identified by the LRA.

2: Which awards have been allocated to an institution? In: Institution name Out: Award title, amount, date, person name, agency name This is a multiple-source query that fetches required and optional attributes from three different entities (grant, agency, and investigators), based on the institution name. Although the grant entities are provided by three data sources (*NSF*, *ANDS*, and *ORCID*), one of the data sources does not associate an institution to the grant (*ORCID*). The experts and the LRA determined the same two service operations as *SignificantMatch*.

3: Which awards have been allocated to a person? In: Person name, ORCID id Out: Award title, amount, date, agency name This is a multiple-operation query, since some of its composition chains have more than one operation. Initially, the experts identified three composition chains as *SignificantMatch*, and two chains as *PossibleMatch* due to increased transmission costs. In particular, the experts found three equivalent chains using *ORCID*, but determined that one of them was more desirable than the other, due to the expected lower size of the data to be transmitted. The LRA method identified two *SignificantMatch* and one *PossibleMatch* composition chains.

4: Which are the publications of an person? In: Person name, ORCID id, year range Out: Publication title, date, DOI, abstract, link, collection name (journal) This is an inference query with optional attributes, since it retrieves data from two entities. The experts identified nine composition chains as *SignificantMatch*, but two of them were incorrect, since they did not provide the entire required output. The LRA method returned seven *SignificantMatch* and one *InsignificantMatch* composition chains.

The date range restriction in the query is expressed as a post filter, executed after receiving the response. In the future, we plan to associate a relational operator with input parameters in the description, wherever it is possible, in order to improve performance and coverage where responses are truncated.

5: Which articles have been published in a specific venue? In: Venue name, date range Out: Same as 4. This is similar to Query 4, also an *inference* query, but uses the venue name as the input, which changes the returned composition chains. The experts identified six *SignificantMatch* composition chains, but one of the chains was incorrect, due to the lack of required inputs for the operation. The experts noted that two of the *PubMed* chains are mutually exclusive, since the overhead and data they return can be considered equivalent. The LRA method returned four composition chains, corresponding to all the correct chains identified by the experts, but including only one of the *PubMed* chains, which was the expected result, due to their mutual exclusivity.

6: Which/How many publications have cited a publication? In: Publication title Out: Citing publication title, date, authors, and venue name/Citation count This is a negative control, because it is not supposed to result in valid composition chains. Accordingly, the experts and the LRA method did not find any results for this query.

7: Which/How many publications have cited the publications of an author? In: Author name, citing publication date. Out: Same as 6 This is similar to Query 6, but uses the author name as the input.

8: Which are the coauthors of a person? In: Person name Out: Coauthor name, affiliation, and publication title This is an isomorphic query, since the inputs and outputs of the query are of the same type. In the same way as Query 6, this query uses independent entities of the same type in the input and output, but in this case, connected through an intermediate entity, a publication. The experts identified seven *SignificantMatch* composition chains, while the LRA method identified eight chains, including the seven chains determined by the experts among the top results. When the experts were confronted with the additional chain reported by LRA, they realized they failed to consider patents as documents. Accordingly, the experts classified the chain as *SignificantMatch*.

8.3 Discussion

We analyzed our results in terms of the traditional F_1-*measure* [42]. Additionally, since all the composition chains are not of equal relevance, we also use the *normalized discounted cumulated gain* (nDCG) [22] to reward retrieval of highly relevant items early and penalize irrelevant chains.

Table 3 Runtime and retrieval performance

Query			Time per exploration length (ms.)				F-measure	nDCG	Max. length
Id	# Triples	# Joins	1	2	3	4			
1	11	3	5	22	209	2185	1.00	1.00	1
2	10	3	15	86	841	37413	1.00	1.00	1
3	12	3	17	187	2791	110150	0.75	0.75	2
4	14	3	21	231	1085	22637	0.94	0.92	3
5	13	3	20	32	63	137	1.00	1.00	2
6a	9	4	12	25	125	490	1.00	1.00	0
6b	5	2	11	92	930	29836	1.00	1.00	1
7a	7	4	17	78	390	1984	1.00	1.00	0
7b	12	3	16	124	1315	36359	1.00	1.00	2
8	15	5	21	69	196	486	1.00	1.00	2

Thus, we assigned gain values of $\{3, 1, -1, -2\}$ to the four-level scale, only considering the first two levels in the ideal gain. Table 3 shows the retrieval correctness results from the evaluation, as measured by the F_1-measure with the binary scale, and by the nDCG with the four-level scale.

The runtime performance was measured on an Intel Core i7 (2.00GHz) machine with 8 GB RAM running Ubuntu 14.04. Results for each query are shown in Table 3, presenting a set of columns with the elapsed time at different exploration lengths. Each measurement corresponds to the average time, in milliseconds, of seven runs of the same kind of query, where the first two runs were discarded. The last columns indicate the F_1-measure, nDCG, and the maximum length of the composition chains found when the exploration length $l = 4$.

None of the returned composition chains was judged as *NoMatch*, which indicates the high quality of the chains produced by the LRA. Additionally, the three errors and two omissions of the experts demonstrate that this task is complex and error-prone, even for experienced developers, which amplifies the potential impact of automating this task. The LRA method successfully identified 28 *SignificantMatch* chains, out of the 30 detected by the experts, only returning 2 *PossibleMatch* and 1 *InsignificantMatch*. Considering only *SignificantMatch* chains, the LRA method showed an average F_1-measure of 0.93, showing a similar performance as the expert judges before they were confronted with the chains produced by the LRA method, with an average F_1-measure of 0.91.

Although not directly comparable to other approaches due to the differences in the test dataset, when contrasted with approaches such as [5,14,54], that use JGDEval, a similar dataset in terms of size, our approach is superior, exhibiting a significantly higher accuracy, with a high average nDCG of 0.96. Unlike the results presented in [26], where the choice of the relevance scale (binary or graded) influenced the results drastically, in our evaluation we found that our scale definitions provided similar results, indicating the stability of our scales. The stability is consequence of the high accuracy, and we also hypothesize that it may be due to the expertise of the relevance judges, which were not only experts in technical aspects, but also experienced in the specific domain of interest, while other approaches typically involve only technical experts for their relevance judgments. Additionally, the approaches in JGDEval Benchmark [26] averaged a running time of 998 milliseconds for service discovery, and our approach performs significantly better, averaging a running time of 15 milliseconds ($l = 1$) and 896 milliseconds for composition chains of length $l = 3$. Although some queries may look similar, such as Query 2 and 3, which only differ in their input parameters, their runtime performance is significantly different, due to the exploration branches that cannot be filtered out, given the presence of some of the input parameters. Overall, the time performance of the queries is aligned with the results of Baccar et al. [2], in what they call reasonable limits for realistic composition scenarios. This performance was enabled by the blocking function and pruning, which could reduce the number of pairwise comparisons to less than 20% for $l = 3$ and less than 1% for $l = 4$, when compared to the brute-force approach that considers every permutation.

Despite the negative implications of an exponential worst-time complexity, in practice the length of the composition chains tends to be low (the distribution of relevant chains is 75% and 25% for lengths 1 and 2, respectively), which resembles the *broken telephone effect*, where long message transmission chains are typically discouraged because they lead to an increased expenditure of resources and error accumulation. Dalvi et al. [7] showed that a large number of data sources are not necessary for a significant coverage of a particular domain; this implies that in order to have good coverage of a domain, the number of service operations n will be rather small, in which case, our approach becomes

practical, especially since our implementation can further be optimized.

We believe that this work should encourage other approaches to make conclusions based on realistic data, rather than on the basis of reflection on ad hoc examples, as has been the case with many recent approaches. For this reason, it is important to create realistic service collections that reflect the current state of the API ecosystem and can be used to compare the performance of diverse approaches, and we think the LRA Web API collection is a good starting point.

9 Conclusions

In this paper, we formalize the LRA approach for automatically composing and invoking REST APIs. The key contribution of our work is a formal model for specifying the semantics of REST APIs, using basic SPARQL graph patterns; the graph pattern nodes correspond to the input and output data types of the API operations and their edges correspond to the semantic relationships between them. Grounded on this formal description model (Sect. 4), we present a fully automated method for service composition and enactment, based on iterative subgraph isomorphism. The LRA service composition process involves the discovery of API operations relevant to the input SPARQL query (Sect. 5) and the traversal of these APIs through a depth-limited search in their data production–consumption relationships, reducing the search space with a set of heuristic rules (Sect. 6). At runtime, the semantic API specifications mediate the lifting of the data produced by the invoked APIs to the shared semantic model and its translation to the syntactic formats required by the consumer APIs (Sect. 7).

The second important contribution of this paper is a collection of semantically annotated real-world Web APIs and a corresponding set of composition queries developed by a domain expert. These queries correspond to realistic use cases, with which our domain expert had experience. We put this dataset forward as a potential benchmark for evaluating the effectiveness of future semantic service composition approaches. This contribution was motivated after a comprehensive survey of available datasets, detailed in Sect. 8, that revealed several deficiencies and lack of availability of previously used service test collections.

Using this benchmark, we evaluated the LRA composition method. Our results validate the accuracy and good performance of the LRA approach.

Our future avenues of investigation include the development of tools to assist in the semantic annotation of Web services. Currently, LRA services are encoded by manually enriching the syntactic service description with graph patterns that represent the schema of the data provided by the service. This labor-intensive task was identified as one of the causes of failure of semantic Web services in the past. However, the syntactic description of services, which includes definitions in natural language, can be used to infer ontology relations between inputs and outputs, in order to promote those descriptions into LRA service descriptions, which would simplify the development life cycle with semantically enhanced Web services.

Acknowledgements This work was financially supported by Natural Sciences and Engineering Research Council.

References

1. Angles R, Gutierrez C (2008) The expressive power of SPARQL. In: The semantic web—ISWC, pp 114–129
2. Baccar S, Rouached M, Verborgh R, Abid M (2018) Declarative web services composition using proofs. Serv Oriented Comput Appl 12:371–389
3. Brickley D, Miller L (2012) FOAF vocabulary specification 0.98. Namespace document, vol 9
4. Brüeckmann T, Gruhn V, Koop W, Ollesch J, Pradel L, Wessling F, Benner M (2017) Codeless engineering of service mashups an experience report. In: Proceedings of the fourteenth IEEE international conference on services computing 2017 (SCC 2017)
5. Cabral L, Domingue J (2012) Ontology based discovery of semantic web services with IRS-III. Springer, Berlin, pp 191–202
6. Corson-Rikert J, Mitchell S, Lowe B, Rejack N, Ding Y, Guo C (2012) The VIVO ontology. Synthesis lectures on semantic web: theory and technology, p 3
7. Dalvi N, Machanavajjhala A, Pang B (2012) An analysis of structured data on the web. Proc VLDB Endow 5(7):680–691
8. D'Arcus B, Giasson F (2009) Bibliographic ontology specification. http://bibliontology.com/specification
9. DCMI: DCMI metadata terms
10. Facebook (2017) GraphQL: a query language for your API. http://graphql.org/
11. Feier C, Polleres A, Dumitru R, Domingue J, Stollberg M, Fensel D (2005) Towards intelligent web services: the web service modeling ontology (WSMO)
12. Gallego MA, Fernández JD, Martínez-Prieto MA, de la Fuente P (2011) An empirical study of real-world SPARQL queries. In: USEWOD workshop
13. García JM, Ruiz D, Ruiz-Cortés A (2012) Improving semantic web services discovery using SPARQL-based repository filtering. Web Semant Sci Serv Agents World Wide Web 17:12–24
14. Gawinecki M, Cabri G, Paprzycki M, Ganzha M (2010) WSColab: structured collaborative tagging for web service matchmaking. In: WEBIST (1), pp 70–77
15. Guha RV, Brickley D, Macbeth S (2016) Schema.org: evolution of structured data on the web. Commun ACM 59(2):44–51
16. Hernández MA, Stolfo SJ (1995) The merge/purge problem for large databases. In: ACM Sigmod record, ACM, vol 24, pp 127–138
17. Heß A, Johnston E, Kushmerick N (2004) ASSAM: a tool for semi-automatically annotating semantic web services. In: The semantic web—ISWC 2004, Springer, pp 320–334
18. Heß A, Johnston E, Kushmerick N (2004) Assam: a tool for semi-automatically annotating semantic web services. In: International semantic web conference, Springer, pp 320–334
19. Hobold G, Siqueira F (2012) Discovery of semantic web services compositions based on SAWSDL annotations. In: International conference on web services (ICWS), IEEE

20. Huang J, Zhou Y, Duan Q, Xing C (2017) Semantic web service composition in big data environment. In: IEEE global communications conference 2017 (GLOBECOM 2017)
21. Jaeger MC, Rojec-Goldmann G, Liebetruth C, Mühl G, Geihs K (2005) Ranked matching for service descriptions using OWL-S. In: Kommunikation in Verteilten Systemen (KiVS), Springer, pp 91–102
22. Järvelin K, Kekäläinen J (2002) Cumulated gain-based evaluation of IR techniques. ACM Trans Inf Syst (TOIS) 20(4):422–446
23. Klusch M, Fries B, Sycara K (2009) OWLS-MX: a hybrid semantic web service matchmaker for OWL-S services. Web Semant Sci Serv Agents World Wide Web 7(2):121–133
24. Klusch M, Kapahnke P, Fries B, Khalid MA, Vasileski M (2010) OWLS service retrieval test collection. http://projects.semwebcentral.org/projects/owls-tc/
25. Kopecky J, Vitvar T, Bournez C, Farrell J (2007) SAWSDL: semantic annotations for WSDL and XML schema. Internet Comput IEEE 11(6):60–67
26. Küster U (2011) An evaluation methodology and framework for semantic web services technology. Ph.D. thesis, Jena, Univ., Diss., 2010
27. Kuster U, Konig-Ries B (2007) Semantic service discovery with DIANE service descriptions. In: 2007 IEEE/WIC/ACM international conferences on web intelligence and intelligent agent technology—workshops
28. Küster U, König-Ries B (2008) Evaluating semantic web service matchmaking effectiveness based on graded relevance. In: Proceedings of the second international conference on service matchmaking and resource retrieval in the semantic web, vol 416, pp 32–46. CEUR-WS. org
29. Lanthaler M, Gütl C (2013) Hydra: a vocabulary for hypermedia-driven web APIs. In: LDOW
30. Maleshkova M, Philipp P, Sure-Vetter Y, Studer R (2019) Smart web services (smartws)—the future of services on the web. arXiv:1902.00910
31. Manning CD, Raghavan P, Schütze H et al (2008) Evaluation in information retrieval. Introduction to information retrieval, vol 1. Cambridge University Press, Cambridge
32. Martin D, Burstein M, Hobbs J, Lassila O, McDermott D et al. (2004) OWL-S: semantic markup for web services. W3C Member Submission, vol 22, p 2007–04
33. Michel F, Faron-Zucker C, Corby O, Gandon F (2019) Enabling automatic discovery and querying of web APIs at web scale using linked data standards. In: Companion proceedings of the 2019 world wide web conference, ACM, pp 883–892
34. Michel F, Zucker CF, Gandon F (2018) SPARQL micro-services: lightweight integration of web APIs and linked data. In: LDOW 2018-linked data on the web, pp 1–10
35. Mouhoub ML, Grigori D, Manouvrier M (2015) LIDSEARCH: a SPARQL-driven framework for searching linked data and semantic web services. In: Gandon F, Guéret C, Villata S, Breslin J, Faron-Zucker C, Zimmermann A (eds) The semantic web: ESWC 2015 satellite events. Springer, Cham
36. Mouhoub ML, Grigori D, Manouvrier M (2017) Towards an automatic enrichment of semantic web services descriptions. In: OTM confederated international conferences on the move to meaningful internet systems, Springer, pp 681–697
37. Paolucci M, Kawamura T, Payne TR, Sycara K (2002) Semantic matching of web services capabilities. In: International semantic web conference, Springer, pp 333–347
38. Patil AA, Oundhakar SA, Sheth AP, Verma K (2004) METEOR-S web service annotation framework. In: International conference on World Wide Web, ACM
39. Pérez J, Arenas M, Gutierrez C (2009) Semantics and complexity of SPARQL. ACM Trans Database Syst (TODS) 34(3):16
40. Petrie CJ, Margaria T, Lausen H, Zaremba M (2008) Semantic web services challenge: results from the first year, vol 8. Springer, Berlin
41. Rietveld L, Verborgh R, Beek W, Vander Sande M, Schlobach S (2015) Linked data-as-a-service: the semantic web redeployed. In: European semantic web conference, Springer, pp 471–487
42. Rijsbergen CJV (1979) Information retrieval, 2nd edn. Butterworth-Heinemann, Newton
43. Rodriguez-Mier P, Pedrinaci C, Lama M, Mucientes M (2016) An integrated semantic web service discovery and composition framework. IEEE Trans Serv Comput 9(4):537–550
44. Roman D, Kopeckỳ J, Vitvar T, Domingue J, Fensel D (2015) WSMO-Lite and hRESTS: lightweight semantic annotations for web services and RESTful APIs. Web Semant Sci Serv Agents World Wide Web 31:39–58
45. Rosenthal R, Rosnow RL (2008) Essentials of behavioral research: methods and data analysis, vol 3. McGraw-Hill, New York
46. Schmachtenberg M, Bizer C, Paulheim H (2014) State of the LOD cloud 2014. University of Mannheim, Data and Web Science Group [en ligne], vol 30
47. Serrano D, Stroulia E, Lau D, Ng T (2017) Linked REST APIs: a middleware for semantic REST API. In: IEEE international conference on web services 2017 (ICWS 2017)
48. Speiser S, Harth A (2011) Integrating linked data and services with linked data services. In: The semantic web: research and applications, Springer
49. Taheriyan M, Knoblock CA, Szekely P, Ambite JL (2012) Rapidly integrating services into the linked data cloud. In: ISWC 2012, Springer, pp 559–574
50. Tomaszuk D (2016) Inference rules for RDF(S) and OWL in N3Logic. arXiv:1601.02650
51. Tsetsos V, Anagnostopoulos C, Hadjiefthymiades S (2006) On the evaluation of semantic web service matchmaking systems. In: 4th European conference on web services, 2006. ECOWS'06, IEEE, pp 255–264
52. Ullmann JR (1976) An algorithm for subgraph isomorphism. J ACM (JACM) 23(1):31–42
53. Verborgh R, Steiner T, Van Deursen D, Van de Walle R, Vallés JG (2011) Efficient runtime service discovery and consumption with hyperlinked RESTdesc. In: 2011 7th international conference on next generation web services practices (nwesp), IEEE, pp 373–379
54. Wei D, Wang T, Wang J, Bernstein A (2011) SAWSDL-iMatcher: a customizable and effective semantic web service matchmaker. Web Semant Sci Serv Agents World Wide Web 9(4):402–417

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Service Oriented Computing & Applications is a copyright of Springer, 2020. All Rights Reserved.